An Empirical-Based Security Benchmarking Framework for Container Confinement

by

Huzaifa Patel

A thesis submitted to the Faculty of Graduate and Postdoctoral Affairs
in partial fulfillment of the requirements for the degree of

Master
in
Computer Science

Carleton University
Ottawa, Ontario

© 2025 Huzaifa Patel

Nomenclature

VM virtual machine. 2

Chapter 1

Background

The existance of cloud computing can be attributed to virtualization, which is a technology that makes it possible for multiple different operating systems (OSs) to run concurrently, and in an isolated environment on the same hardware. How does virtualization achieve this? It makes use of a machines hardware to support the software that creates and manages virtual machines (VMs). A virtual machine (VM) is a virtual environment that provides the functionality of a physical computer by using its own virtual central processing unit (VCPU), memory, network interface, and storage. The software that creates and manages VMs is formally called a hypervisor or virtual machine monitor (VMM). The virtualization marketplace is comprised of four notable hypervisors, which are: (1) VMWare, (2) Xen, (3) Kernel-based Virtual Machine (KVM), and (4) Hyper-V. The operating system running a hypervisor is called the host OS, while the VM that uses the hypervisor is called the guest OS. To maintain consistency within this proposal, we will use the word "hypervisor" when referring to virtualization software. Hypervisors can be divided into two types, depending on where they are located on the machine: (1) type 1 and (2) type 2.

1.0.1 Type 1 Hypervisor

Type 1 hypervisors run directly on physical hardware to create, control, and manage VMs, and do not require support from the host OS. Type 1 hypervisors are also called native or bare-metal hypervisors. The first hypervisors, which IBM developed in the 1960s were type 1 hypervisors [?]. Examples of type 1 hypervisors include, but are not limited to Xen, KVM, VMware ESX, and Microsoft Hyper-V [?].

1.0.2 Type 2 Hypervisor

Type 2 hypervisors (also called hosted hypervisors) are installed on the OS (Windows, Linux, MacOS), similar to how computer programs are installed onto an OS. In other words, a type 2 hypervisor runs as a process on the host OS. Type 2 hypervisors abstract guest OSs from the host OS by introducing a third software layer above the hardware, as shown in figure 2.1. Examples of type 2 hypervisors include but are not limited to VMware Workstation, VirtualBox, and QEMU [?].

1.0.3 Problems With Type 1 & Type 2 Hypervisor Classifications

Although the definitions of type 1 and type 2 hypervisors are widely accepted, there are gray areas where the distinction between the two remain unclear. For instance, according to its creator RedHat, KVM is implemented and deployed using two Linux kernel modules that effectively convert the host Linux OS into a type-1 hypervisor [?]. At the same time, KVM can be categorized as a type 2 hypervisor because KVM VM's are standard Linux processes that are competing with other Linux processes for CPU time given by the Linux kernel's native CPU scheduler [?]. Due to disagreements and vagueness in the definitions of type 1 and type 2 hypervisors, two new classifications were defined: (1) native hypervisors and (2) emulation hypervisors [?].

1.0.4 Native Hypervisor

Native hypervisors are hypervisors that push VM guest instructions directly to the physical machines CPU, by using virtualization extensions like Intel VT-x. [**?**]. Examples of Native hypervisors include but are not limited to Xen, KVM, VMware ESX, and Microsoft HyperV.

1.0.5 Emulation Hypervisor

Emulation hypervisors are hypervisors that emulate every VM guest instruction using software virtualization [**?**]. Emulated guest instructions are very easy to trace because all guest VM instructions are trapped to the hypervisor. Examples of emulation hypervisors include but are not limited to QEMU, Bochs, early versions of VMware-Workstation, and VirtualBox [**?**].

1.0.6 Why do we have run Docker with sudo?

By default, Docker's daemon (dockerd) runs as root, and access to it is controlled through the Unix socket:

Copy Edit /var/run/docker.sock This socket is owned by root and typically part of the docker group. If your user isn't in the docker group, you'll need sudo to interact with it.

Why Docker Needs Elevated Privileges: Container Isolation Requires Kernel Features:

Managing cgroups, namespaces, mounting filesystems, setting up virtual networks—all require elevated permissions.

By Default, Docker Protects Access to Its Control API:

docker.sock is a privileged control interface that allows full system control (you can effectively become root via it).

You can check if Yama is enabled by inspecting the ptrace_scope value in /proc:

bash Copy Edit cat /proc/sys/kernel/yama/ptrace_scope What the Values Mean: Value Description 0 Yama is disabled (classic ptrace behavior). 1 Yama is enabled in restrictive mode (processes can only ptrace their children). 2 Yama is enabled, requires CAP_SYS_PTRACE to trace any process. 3 Yama is locked down; no ptrace allowed regardless of capabilities.